Факултет за информатички науки и компјутерско инженерство

Проектна задача

Предмет:

Вештачка интелигенција за игри

Тема:

Имплементација на A* пребарување на патека и стратегии за избор на цел во tower defense игра

Tower Defense

Изработил:
Ангел Мирчески - 216131

Ментор:
Проф. д-р Андреа Кулаков

Скопје

октомври 2026

1. Вовед

Tower Defense 2D е прототип на игра од типот „одбрана на кула“ (tower defense) со едно ниво, изработен во Unity по предметот „Вештачка интелигенција за игри“. Играчот гради кули на мрежа од 16 × 10 ќелии за да ги запре непријателите што одат од левиот кон десниот раб. Проектот прикажува како два класични методи од вештачката интелигенција — пребарување на патека со A* и избор на цел — ја обликуваат играта.

Главната цел е да се изгради разбирлив систем во кој непријателите ја наоѓаат најкратката патека со сопствена имплементација на алгоритамот A*, ја пресметуваат одново при секое градење кула, а кулите бираат цел според заменливи стратегии (First, Closest, Strongest, Weakest). Ефектот од пребарувањето на патека е и измерен: автоматски играчи (ботови) го споредуваат градењето кули покрај права патека со градење ѕидови што ја продолжуваат патеката.

2. Основни информации

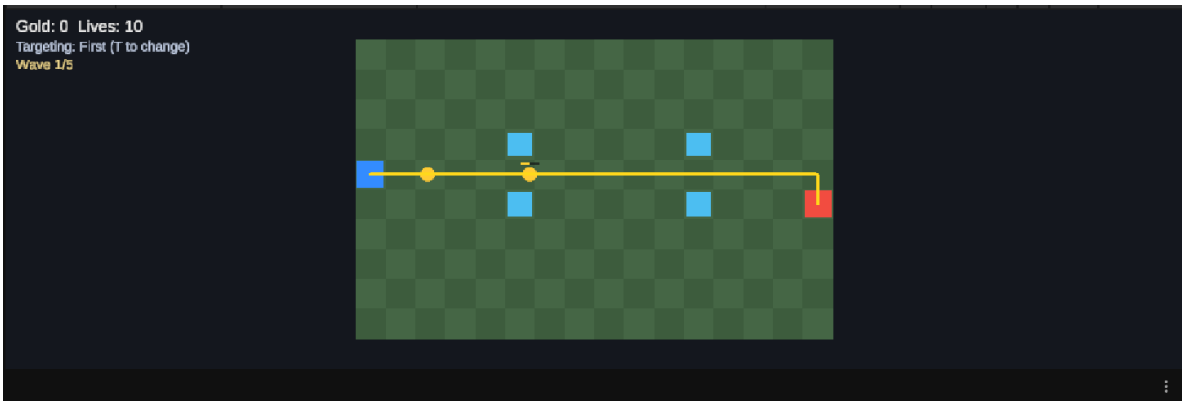
Unity верзија	6000.5.2f1 (Unity 6)
Тип на игра	2D tower defense, едно ниво
Главна сцена	Assets/Scenes/TowerDefense.unity — се создава со командата Tower Defense > Build Level
Рендерирање и влез	Universal Render Pipeline (2D Renderer); Input System (нов)
Главни системи	A* пребарување, повторно пребарување, стратегии за избор на цел, бранови, проектили, состојби на играта
Изворен код	github.com/mircheskiangel/TowerDefence

Не се користат надворешни ресурси (assets) ниту платени пакети. Сите спрајтови се генерираат во код.

3. Играње на играта

Играчот започнува со 100 злато и 10 животи. Непријателите се појавуваат на ќелијата на левиот раб и одат кон целта на десниот раб. Задачата е да се издржат сите пет бранови со кули поставени на слободни ќелии.

Лев клик на слободна ќелија	Поставување кула (зелено = дозволено, црвено = недозволено)
Глувче над ќелија или кула	Приказ на дострелот (range) на кулата
P	Прикажување или криење на линијата на моменталната A* патека од појавата до целта
T	Менување на режимот на избор на цел за сите кули: First → Closest → Strongest → Weakest
Space	Прескокнување на одбројувањето пред следниот бран
R (само додека е прикажан крајниот панел)	Рестартирање на нивото
Копче Restart (крајниот панел)	Рестартирање на нивото



Слика 1. Главната сцена TowerDefense во Play режим.

4. Основен циклус на играта

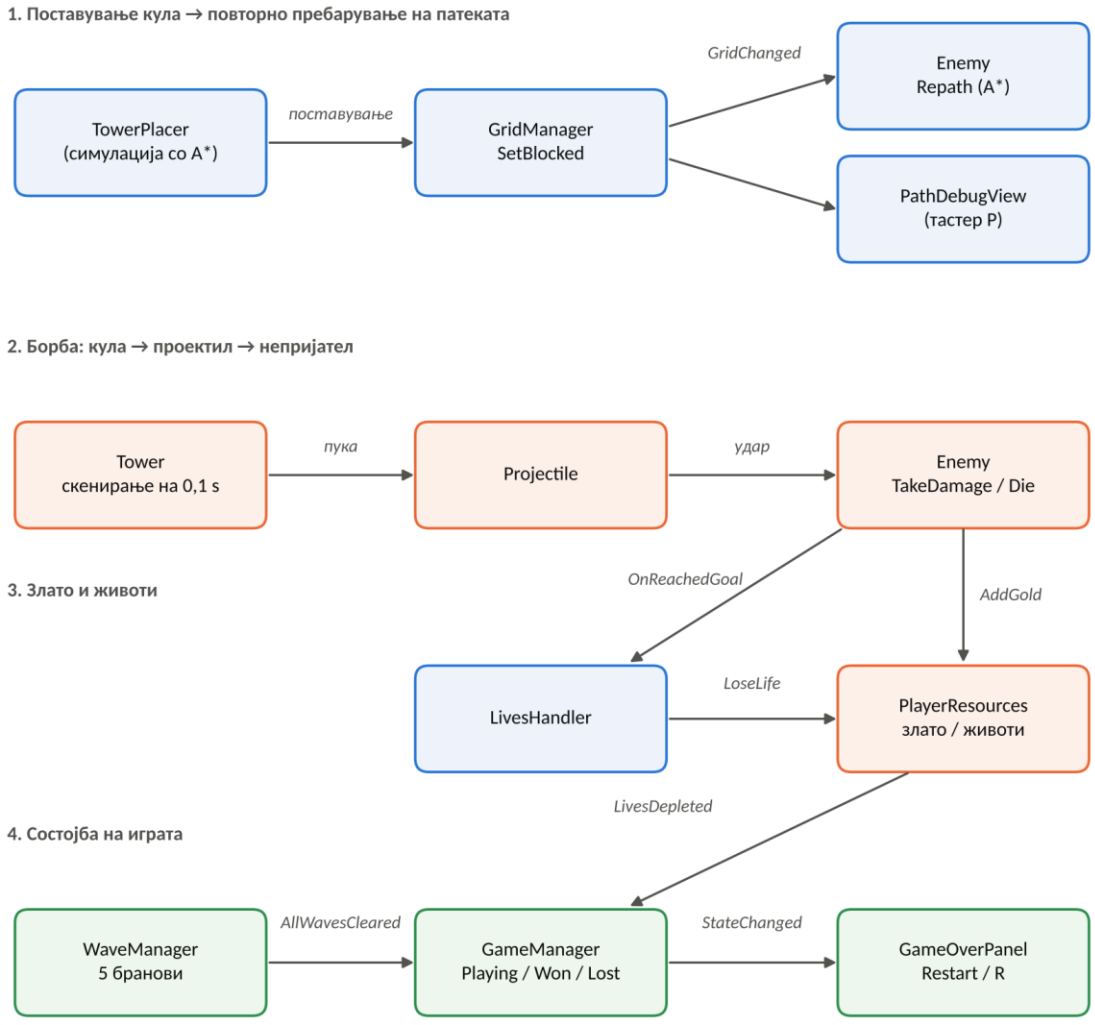
1. Играчот започнува со 100 злато и 10 животи; пред првиот бран има одбројување од 10 секунди (8 секунди меѓу брановите).
2. Во текот на одбројувањето и брановите играчот поставува кули (25 злато) на слободни ќелии.
3. Непријателите се појавуваат на ќелијата за појава и одат по најкратката A* патека до целта.
4. Секоја поставена кула ја менува мрежата, па сите непријатели одново ја пресметуваат патеката од ќелијата на која се наоѓаат.
5. На секои 0,1 секунди кулите бираат цел во дострел и пукаат проектили; убиен непријател носи злато (Grunt 4, Tank 10).
6. Непријател што ќе стигне до целта одзема еден живот и не носи злато.
7. Бранот е завршен кога сите негови непријатели се појавени и ниеден не е жив. По петтиот бран играта завршува со порака „YOU WIN!“, а при 0 животи со „GAME OVER“.

На крајот играта се замрзнува и се прикажува панел со копче Restart.

5. Архитектура

Целиот код за извршување е во Assets/Scripts под именскиот простор TowerDefense. Системите меѓусебно комуницираат претежно преку C# настани (events), па ниту една класа не мора постојано да „пита“ друга.

Папка	Главни класи
Core	GameManager (состојби Playing / Won / Lost, рестарт), PlayerResources (злато, животи), LivesHandler, CameraFitter
Grid	GridManager (ќелии, претворање ќелија ↔ светски координати, блокирани ќелии, појава и цел)
Pathfinding	AStarPathfinder, MinHeap (бинарна купа), PathDebugView (тастер P)
Enemies	Enemy (движење, повторно пребарување, здравје), EnemyRegistry, EnemyHitFlash, EnemyHealthBar
Towers	Tower (скенирање, пукање), TowerPlacer (поставување + проверка за блокирана патека), Projectile
Towers/Targeting	TargetingMode, ITargetSelector, TargetSelectors (четирите стратегии), TargetingSettings, TargetingModeSwitcher
Waves	WaveManager (одбројување, корутина за појава, проверка дали бранот е завршен)
UI	ResourcesHUD, TargetingHUD, WaveHUD, MessageDisplay, GameOverPanel
Data	ScriptableObject-и: EnemySO, TowerSO, WaveSO
Assets/Editor	LevelBuilder (го гради нивото), BalanceBot (автоматски тестови за баланс, само во уредникот)



Слика 2. Тек на настаните меѓу главните класи (сино: градење и пребарување на патека; портокалово: борба и ресурси; зелено: состојба на играта).

Главните настани и кој ги користи:

- `GridManager.GridChanged` → секој `Enemy` одново пребарува патека, `PathDebugView` се исцртува одново, `TowerPlacer` го чисти својот кеш за истакнување.
- `Enemy.OnReachedGoal` → `LivesHandler` повикува `PlayerResources.LoseLife()`.
- `Enemy.OnDied` → се активира еднаш по убиство (златото го додава самиот непријател); го користат ботовите за броење убиства.
- `PlayerResources.Changed` → `ResourcesHUD` ја освежува ознаката „Gold / Lives“.
- `PlayerResources.LivesDepleted` → `GameManager` преминува во Lost, `WaveManager` ја запира појавата.
- `WaveManager.AllWavesCleared` → `GameManager` преминува во Won.
- `GameManager.StateChanged` → `GameOverPanel` го прикажува крајниот екран.

- `TargetingSettings.ModeChanged` → секоја `Tower` веднаш повторно скенира, `TargetingHUD` се освежува.

6. Правила на играта

- **Мрежа:** 16 × 10 ќелии, 1 светска единица по ќелија. Ќелијата за појава е на левиот раб (сина, ќелија (0,5)), целта на десниот раб (црвена, ќелија (15,4)).
- **Злато:** играчот започнува со 100 злато. Кулата чини 25 злато. Убиен непријател носи злато (Grunt 4, Tank 10).
- **Животи:** 10 животи. Секој непријател што ќе стигне до целта одзема 1 живот и не носи злато.
- **Поставување кули:** кула може да се постави на секоја слободна ќелија освен на појавата и целта, ако играчот има доволно злато. Кулите ја **блокираат** ќелијата, па непријателите мора да ја заобиколуваат.
- **Правило за блокирана патека:** поставување што би оставило **без патека** од појавата до целта, или би заробило непријател што веќе оди, се **одбива**. Не се троши злато и 1,5 секунди се прикажува пораката „Path blocked!“. Поставување кула на ќелија на која стои непријател е дозволено; тој просто излегува од неа.
- **Бранови:** 5 бранови. Пред секој има одбројување (10 s пред првиот, 8 s меѓу брановите). Бранот е завршен кога сите негови непријатели се појавени **и** ниеден не е жив.
- **Победа / пораз:** чистење на петтиот бран е победа („YOU WIN!“ со преостанатите животи), 0 животи е пораз („GAME OVER“ со достигнатиот бран).

Непријател	Здравје	Брзина (ќелии/s)	Злато	Изглед
Grunt	76	2,3	4	мал жолт круг
Tank	400	1,1	10	голем виолетов круг

Basic Tower	Цена	Дострел	Штета	Брзина на пукање	Брзина на проектил
	25	2,5 ќелии	10	1,5 истрели/s	8 ќелии/s

Бран	Непријатели
1	5 Grunt, еден на 1,5 s
2	8 Grunt, еден на 1,2 s
3	6 Grunt на 1,2 s, потоа (по 2 s) 2 Tank на 3 s
4	10 Grunt на 1,0 s, потоа (по 2 s) 4 Tank на 2,5 s
5	12 Grunt на 0,9 s, потоа (по 2 s) 6 Tank на 2 s

7. A* пребарување на патека

Непријателите ја наоѓаат патеката со сопствена имплементација на A* во AStarPathfinder.cs. Отворената множество е бинарна купа (heap) во MinHeap.cs.

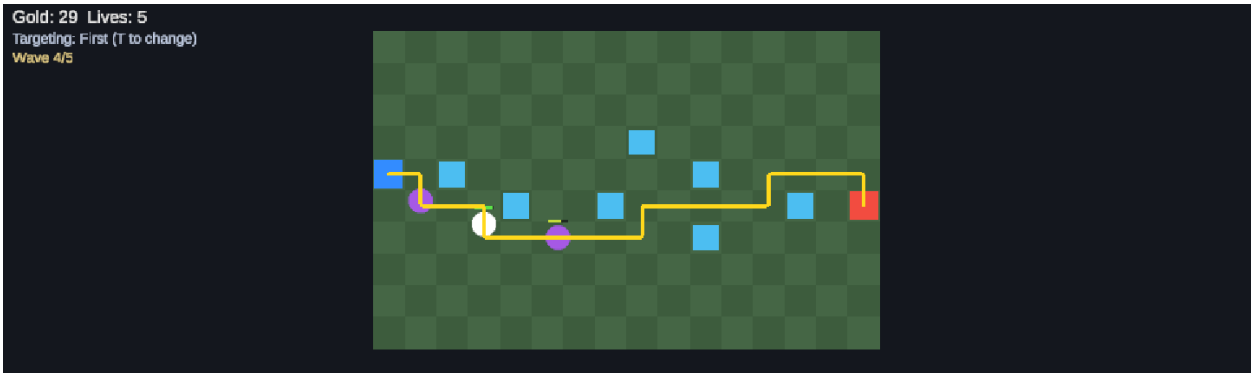
- **Простор на пребарување:** мрежата 16 × 10. Од секоја ќелија е можно движење во **4 насоки** (горе, десно, долу, лево); секој чекор чини 1. Блокираните ќелии (кулите) не се проодни.
- **Оценка:** секоја ќелија добива **F = G + H**. G е бројот на чекори од почетокот, H е проценетиот број преостанати чекори. A* секогаш ја проширува отворената ќелија со најмало F.
- **Хеуристика — Менхетн растојание:** H = |dx| + |dy|. На празна мрежа со 4 насоки тоа е точниот број чекори, а пречките можат само да ја продолжат вистинската патека. Затоа хеуристиката **никогаш не преценува** (допуштлива е, admissible), што гарантира **најкратка** патека, а пребарувањето сепак се влече директно кон целта.
- **Отворено множество како бинарна купа:** дрво зачувано во рамна листа (децата на елементот i се на 2i+1 и 2i+2). Додавањето го „качува“ елементот нагоре, а вадењето на најдобриот ја „спушта“ последната ставка од коренот. Двете операции се **O(log n)** наместо скенирање на целата листа.
- **Решавање на еднакви оценки по H:** при исто F предност има ќелијата со помало H (поблиску до целта). Така се прошируваат помалку ќелии и патеките се понасочени кон целта. На празна мрежа се прошируваат точно 17-те ќелии на патеката.
- **Одложено бришење (lazy deletion):** кога ќе се најде поевтин пат до ќелија, таа се додава повторно со подобра оценка. Постарите, полоши копии се прескокнуваат кога ќе излезат од купата, бидејќи ќелијата е веќе затворена. Така се избегнува скапата операција „decrease key“.
- **Сложеност:** во најлош случај **O(E log V)**. Тука V = 160, а E ≤ 4V, па едно пребарување е многу ефтино.
- **Резултат:** листа од ќелии **од почеток до цел, двете вклучени**, или **null** ако целта е недостижна. Патеката се составува со следење на врските „came from“ наназад од целта.

Проверка во уредникот на Unity со поставени ѕидови:

Случај	Резултат
Празна мрежа	најкратка патека од 17 ќелии; A* проширил само 17 ќелии
Два ѕида со процепи	цик-цак патека од 33 ќелии; проширени 98 ќелии
Запечатен процеп	враќа null по проширување на 50-те ќелии што уште се достижни, без исклучок



Слика 3. Илустрација на A патеката на мрежата 16 × 10 (S = појава, G = цел): а) празна мрежа, б) два ѕида со процепи. Сликата е нацртана посебно, а не е скриншот; користи исти правила (4 насоки, Менхетн растојание) и дава исти должини на патеката (17 и 33 ќелии) како проверката во Unity.*



Слика 4. Жолтата линија на моменталната A патека (тастер P) се менува при градење кули.*

8. Повторно пребарување на патека

Датотека: Enemy.cs.

- Секој непријател е претплатен на `GridManager.GridChanged`. Кога ќе се постави кула, настанот се активира еднаш и **секој непријател што оди одново го извршува A*** од ќелијата на која моментално се наоѓа до целта.
- Непријателот обично е меѓу центрите на две ќелии. Ако веќе се движи кон следната ќелија од новата патека, продолжува напред; инаку прво оди до центарот на својата ќелија (најмногу половина ќелија). **Никогаш не се телепортира.**
- **Почетната ќелија секогаш се смета за проодна**, дури и ако е блокирана: A* не ја проверува почетната ќелија, само нејзините соседи. Затоа непријател што стои на ќелија на која е изградена кула едноставно излегува од неа.
- Ако воопшто нема патека, непријателот застанува во центарот на ќелијата и чека; следниот `GridChanged` предизвикува нов обид. (Со правилото од следното поглавје тоа нормално не се случува.)

9. Одбивање на поставување што ја блокира патеката

Датотека: TowerPlacer.cs.

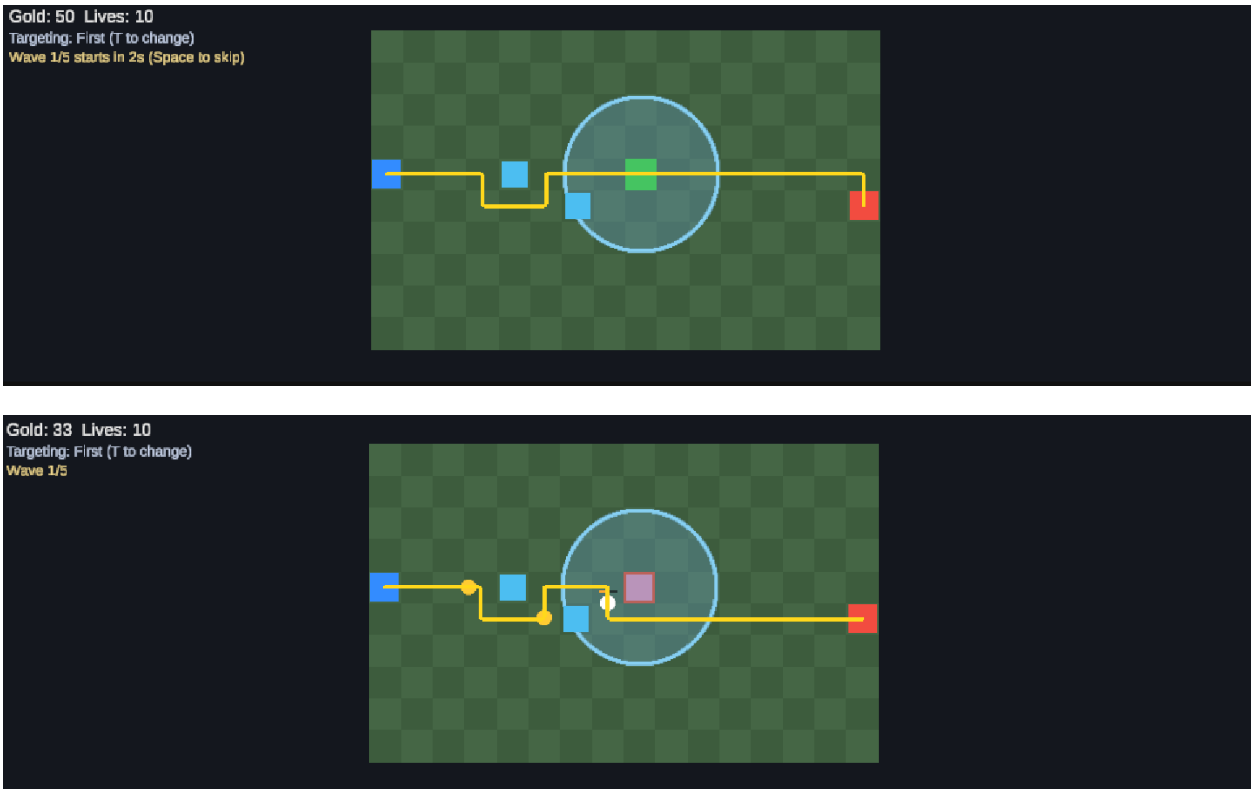
Пред кулата навистина да се изгради, поставувањето се **симулира**:

1. Ќелијата се означува како блокирана со `GridManager.SetBlockedSilently`, која **не** го активира `GridChanged`.
2. A* се извршува од **појавата** до целта и од **моменталната ќелија на секој непријател што оди** до целта (со истото правило „почетната ќелија е секогаш проодна“). Непријателите што веќе чекале без пат пред кликот се занемаруваат.
3. Ќелијата се враќа на старата вредност во `finally` блок, па мрежата секогаш останува точно каква што била.

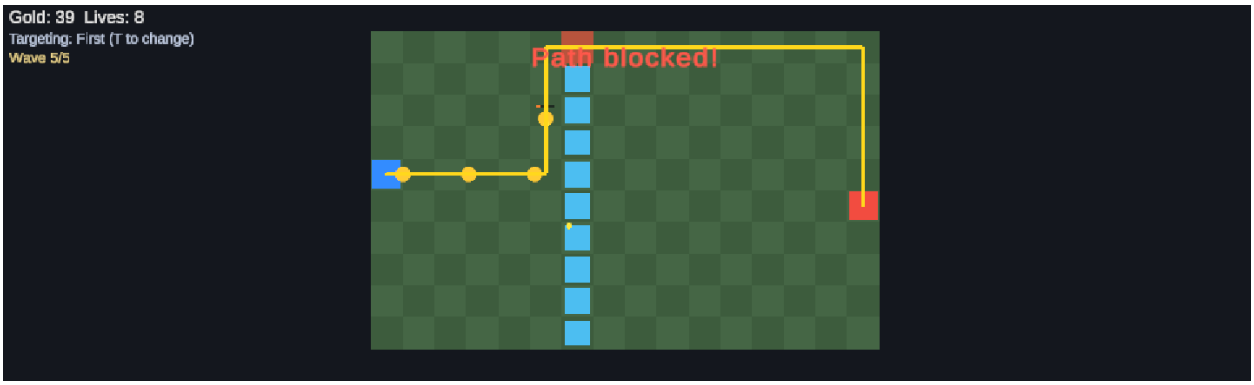
4. Ако некое пребарување не најде патека, поставувањето се одбива: не се троши злато, не се создава кула и се прикажува „Path blocked!“.

Зошто GridChanged не смее да се активира за време на симулацијата: тој настан значи „мрежата навистина се променила“. Ако се активираше, секој непријател би пребарувал одново за кула што не постои, и уште еднаш при враќањето на ќелијата; непријателите би завршувале на беспотребни заобиколувања, а линијата за дебагирање би трепкала. Тивкиот метод го прави експериментот невидлив за остатокот од играта.

Истата симулација го боите истакнувањето под глувчето во црвено над ќелии што би ја блокирале патеката. Таа се извршува само кога се менува ќелијата под глувчето (или кога мрежата навистина се менува), а не секој кадар; при клик секогаш се прави целосна проверка.



Слика 5. Истакнување на ќелијата: зелено кога поставувањето е дозволено, црвено кога не е.



Слика 6. Порака при одбиено поставување.

10. Избор на цел (шаблон Strategy)

Датотеки: Towers/Targeting/ и Tower.cs.

- `ITargetSelector` е интерфејсот на стратегијата: „од непријателите во дострел, кого да нападнам?“. Секој режим е своја мала класа, па кулата само ја повикува моменталната стратегија и нема `switch` по режими. Нов режим значи една нова класа.
- Четирите селектори делат една основна класа што на секој непријател му дава **оценка**, ја чува највисоката и при еднаквост бира **помало растојание до кулата** (па резултатот е секогаш детерминистички).

Режим	Оценка (поголема = подобра цел)
First	PathProgress (најблиску до целта)
Closest	минус растојанието до кулата
Strongest	моменталното здравје
Weakest	минус моменталното здравје

- **PathProgress е минус преостанатата должина на патеката** (0 на целта). Не е поминатото растојание: по повторното пребарување непријател што одел по долго заобиколување може да поминал повеќе, а сепак да е подалеку од целта. Преостанатата должина на моменталната A* патека е вистинската мерка за „најнапред“.
- `TargetSelectors.Get(mode)` враќа **по една споделена инстанца за секој режим**. Селекторите немаат состојба, па ги користат сите кули и за време на играта не се алоцира меморија.
- **Скенирањето е на тајмер од 0,1 s**, не секој кадар: наоѓањето непријатели во дострел споредува секоја кула со секој непријател, а десет пати во секунда наместо секој кадар заштедува поголем дел од таа работа без видливо задоцнување. Секоја кула има **случајно задоцнување на првото скенирање**, за да не скенираат сите кули во ист кадар. Листата на непријатели во дострел се користи повторно, па скенирањето не создава ѓубре.

- Режимот е **глобален** (`TargetingSettings`); со T се менува за сите кули, вклучително и за подоцна поставените, и тие веднаш повторно скенираат.



Слика 7. Кули во борба: проектили и избраниот режим на таргетирање.

11. Безбедно ракување со подвижни цели

Датотеки: Projectile.cs и Enemy.cs.

- **Проектилите ја следат целта** и секој кадар ја паметат нејзината позиција како „последна позната позиција“. Ако целта умре или се уништи додека проектилот е во лет, тој ја отфрла референцата, продолжува кон последната позната позиција и потоа исчезнува **без да нанесе штета**. Живот од 3 секунди го отстранува проектилот во секој друг случај. Преоптовареният (overloaded) оператор == на Unity уништен објект го третира како null, па грешка од типот null-reference не е можна.
- **Непријателите умираат точно еднаш:** првиот смртоносен удар го поставува `IsDead` пред да се случи било што друго (настан, злато, `Destroy`). Секој подоцнежен удар, дури и од проектили што пристигнуваат во ист кадар, се игнорира, па наградата никогаш не се исплаќа двапати. Непријателот веднаш го напушта и `EnemyRegistry`, па ниту една кула не може да го избере при следното скенирање.
- Стигнувањето до целта **не е смрт**: чини живот и не носи злато. Се користи посебен настан `OnReachedGoal`.

12. Состојби на играта и рестарт

Датотеки: GameManager.cs и GameOverPanel.cs.

- **GameManager** е машина со три состојби: Playing, па Won или Lost. Слуша `LivesDepleted` (→ Lost) и `AllWavesCleared` (→ Won). Кога играта ќе заврши, поставува `Time.timeScale = 0`, што ги замрзнува непријателите, кулите, проектилите и одбројувањата, а корисничкиот интерфејс продолжува да работи.
- **Состојбата се менува само еднаш:** секоја промена минува низ еден метод `SetState` што ја игнорира барањето ако моменталната состојба не е Playing. Завршена игра не може да премине од Won во Lost или обратно.
- **Lost победува при истовремен настан:** Lost се применува веднаш, а Won само се запишува и се применува во `LateUpdate` на крајот на кадарот. Ако играчот во истиот кадар ги изгуби и животите, Lost е веќе поставен и барањето за Won се игнорира.
- **Зошто timeScale се враќа на 1 пред повторното вчитување:** `Time.timeScale` е глобална вредност што преживува вчитување на сцена. Без враќање, новото ниво би започнало замрзнато. GameManager го поставува на 1 и во `Awake`.
- **Статичка состојба при исклучен Domain Reload:** `GameManager` се извршува прв (`DefaultExecutionOrder(-1000)`), па неговиот `Awake` го чисти `EnemyRegistry`, ги отстранува претплатниците од `Enemy.OnDied` и `Enemy.OnReachedGoal` и го ресетира режимот на таргетирање. Singleton-ите (`GridManager`, `PlayerResources`) секогаш ја земаат најновата инстанца.

- **GameOverPanel** го прикажува насловот („YOU WIN!“ или „GAME OVER“) и резултатот („Lives left: X“ или „Reached wave X of 5“). Копчето Restart и тастерот R (само додека панелот е видлив) го повикуваат `GameManager.Restart()`.

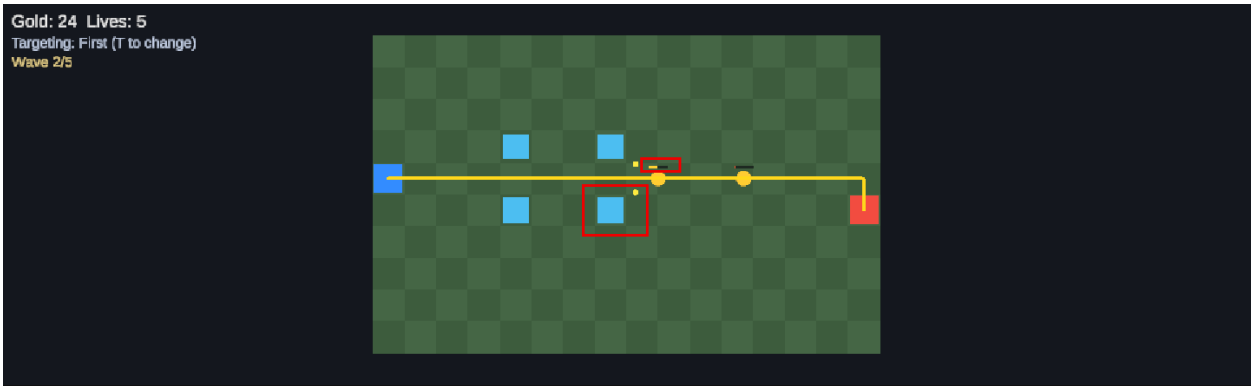


Слика 8. Крајниот панел со копче Restart.

13. Визуелна повратна информација

Следниве ефекти се само визуелни и немаат влијание врз текот на играта:

- **Блесок при удар (EnemyHitFlash):** кога непријателот ќе преживее удар, неговиот спрајт станува бел околу 0,08 секунди. Оригиналната боја се памти еднаш при појавата, па нов удар во текот на блесокот само го рестартира тајмерот и секогаш се враќа на точната боја од EnemySO.
- **Лента за здравје (EnemyHealthBar):** темна позадина и обоено пополнување, поставени над непријателот без Canvas. Лентата ја поништува скалата на непријателот, па има иста големина (0,6 × 0,09 светски единици) и на Grunt и на поголемиот Tank. Бојата преминува од зелена преку жолта во црвена. Скриена е при полно здравје и се ажурира само кога здравјето се менува. Како дете на непријателот, исчезнува заедно со него.
- **Трепет на кулата:** при пукање кулата се зголемува за 10% во траење од 0,05 секунди.
- При смрт непријателите исчезнуваат веднаш, без постепено бледнеење, бидејќи од тоа зависат регистарот и златото.



z

Слика 9. Ленти за здравје и блесок при удар.

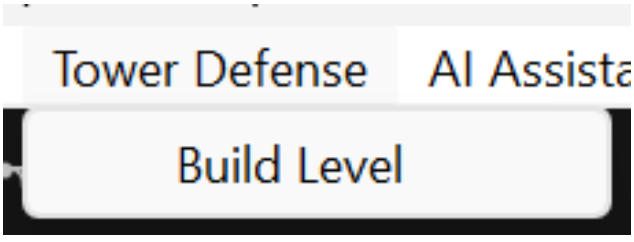
14. Генератор на нивото

Целото ниво го создава една едиторска скрипта, LevelBuilder.cs (Assets/Editor). Таа ги генерира спрајтовите (PNG датотеки пополнети со бели пиксели што се бојат при извршување), материјалот, податочните ресурси (непријатели, кула, бранови), сите prefab-и, хиерархијата на сцената, корисничкиот интерфејс и сите врски меѓу компонентите.

Сцените и prefab-ите **никогаш не се уредуваат рачно**. Така проектот е повторлив и се избегнуваат конфликти во Unity датотеките на сцената: повторното извршување на Build Level не создава дупликати, туку ја гради сцената одново и ги презапишува генерираните ресурси на истото место (GUID-ите се задржуваат, па референците остануваат валидни). Сите бројки за балансот се поставени во LevelBuilder.cs.

Создавање и стартување на нивото

1. Проектот се отвора во Unity Hub со Unity 6000.5.2f1.
2. Од менито се избира **Tower Defense > Build Level**.
3. Со **Play** се стартува играта (сцената Assets/Scenes/TowerDefense.unity ја отвора генераторот).



Слика 10. Командата за градење на нивото.

15. Балансирање и евалуација

Нивото е балансирано со тест алатка само за уредникот, BalanceBot.cs. Во Play режим таа автоматски игра на 4× брзина со четири едноставни стратегии, користејќи **само заработено злато** и **истите правила за поставување** како човек (го повикува `TowerPlacer.TryPlaceTower`):

- **none:** не гради ништо.
- **weak:** гради точно 4 кули во еден куп покрај патеката.
- **reasonable:** троши злато што е можно побрзо на кули во парови од двете страни на патеката.
- **maze:** гради ѕидови со наизменични процепи, така што A* мора да ги води непријателите по подолга патека во облик на змија.

Цели на балансирањето: играч без кули и „слаб“ играч губат; „разумен“ играч победува, но губи неколку животи; „лавиринт“ играчот победува со повеќе преостанати животи од разумниот.

Променети вредности

Вредност	Пред	После
Почетно злато / животи	100 / 10	100 / 10 (непроменето)
Basic Tower	25 злато, дострел 2,5, штета 10, 1,5 истрели/s	непроменето
Grunt: здравје / брзина / злато	30 / 2,0 / 10	76 / 2,3 / 4
Tank: здравје / брзина / злато	120 / 1,0 / 25	400 / 1,1 / 10
Бранови 1–5	како во табелата во поглавје 6	непроменето
Вкупно злато од убиства	710	284

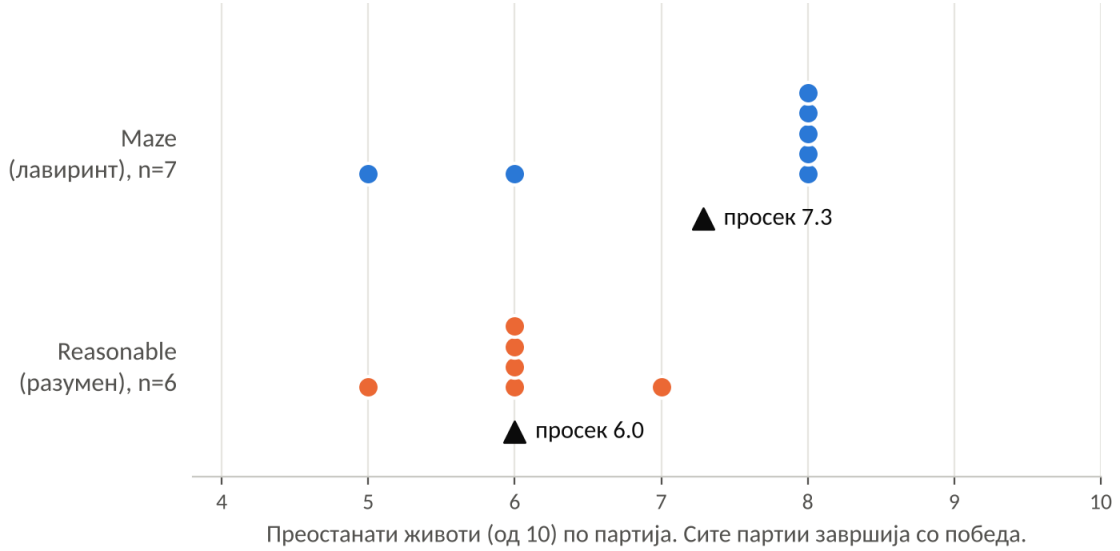
Пред балансирањето сите стратегии освен none победуваа со сите 10 животи (дури и 4 кули беа доволни). Образложение на промените:

- **Награди намалени за 60%:** пред тоа секоја стратегија можеше да си дозволи околу 31 кула. Со 284 злато од убиства играчот вкупно добива околу 13 кули, па секои 25 злато се вистински избор.
- **Grunt потежок и побрз:** при 76 здравје му требаат 8 удари наместо 3, па куп од 4 кули не стигнува од четвртиот бран. Брзина 2,3 го намалува времето во дострел на кулите, што ги казнува кулите покрај права патека, а ги наградува подолгите патеки — токму тоа го користи лавиринтот.
- **Tank како тест за крајот на играта:** со 400 здравје му требаат 40 удари, а се појавува од третиот бран. Тој е заканата што на крајот провлегува покрај обична линија од кули.
- **Непроменето:** почетното злато (100 купува 4 кули веднаш), статистиката на кулата и брановите (поголеми бранови 4–5 главно го урива лавиринтот во крајот).

Резултати (завршна проверка, по 2 партии за стратегија)

Играч	Исход	Животи	Бран	Кули	Злато	Убиства
none	загуба / загуба	0 / 0	2 / 2	0	0	0 / 0
weak	загуба / загуба	0 / 0	4 / 4	4	100	20 / 17
reasonable	победа / победа	7 / 6	5	13	325	50 / 49
maze	победа / победа	5 / 8	5	13 / 14	325 / 350	48 / 51

Преку сите партии на ботовите со конечните вредности, „лавиринт“ играчот остави **8, 8, 6, 8, 8, 5, 8** животи (просек **7,3**), а „разумниот“ **6, 5, 6, 7, 6, 6** (просек **6,0**). Сите овие партии завршија со победа.



Слика 11. Преостанати животи по партија за разумниот и за лавиринт играчот (триаголникот е просекот).

Што покажуваат резултатите

Без кули, или со само мал куп од четири, играчот губи; разумна линија од кули победува, но губи неколку животи, па нивото е совладливо, но не тривијално. Лавиринт играчот во просек победува со повеќе преостанати животи, а троши приближно исто злато: принудувањето на подолга рута со A* ги задржува непријателите подолго во дострел на кулите. Во текот на балансирањето овој ефект се појави дури кога непријателите станаа доволно брзи за **времето во дострел** да биде тесното грло; при бавни непријатели обичните парови кули беа исто толку добри. Затоа повторното пребарување на патека не е само техничка особина, туку вистинска стратегија за играчот.

Тек на балансирањето

Круг	Што се менувало	Исход
Почеток	вредностите од претходниот чекор	сите освен none победуваат со 10/10
1	намалени награди, поголемо здравје на непријателите	weak губи; reasonable и maze уште победуваат со 10/10
2	побрзи и поиздржливи непријатели	reasonable губи рано; maze победува со 8 животи (прв знак на ефектот)
3	притисокот преместен на брановите 4–5	reasonable 10/10; maze 9
4	посилни Tank непријатели	двајцата губат во бран 5
5	брзи Grunt, среден Tank	reasonable на работ; maze губи
6	поставки од круг 2, малку полесно	сите цели исполнети

Ограничувања

- Резултатите на ботовите малку варираат меѓу сесиите, бидејќи првото скенирање на секоја кула започнува по случајно задоцнување; во поединечни партии лавиринт играчот понекогаш остава помалку животи од разумниот, иако е подобар во просек.
- Ботовите се едноставни фиксни редоследи на градење, а не оптимални играчи.
- Едно ниво, еден тип кула и два типа непријатели.
- Движењето е само во 4 насоки и нема летачки непријатели (сè оди по мрежата).

16. Можности за проширување

- **Втор тип кула** (на пример бавна, но со штета на површина), додаден како уште еден `TowerSO` ресурс.
- **Дијагонално движење** со 8 насоки и октилно (или евклидово) растојание како хеуристика, бидејќи Менхетн растојанието би преценувало и би ја нарушило допуштливоста.
- **Поле на тек (flow field)** — едно пребарување од целта низ целата мрежа наместо по еден A* за секој непријател, што подобро се скалира со многу непријатели.
- **Надградби на кулите** (повеќе штета, дострел или брзина за злато).
- **Летачки непријатели** што ја игнорираат мрежата и летаат право кон целта, па лавиринтите не помагаат против нив.
- **Режим на таргетирање по кула** наместо еден глобален режим.

17. Заклучок

Во проектот е реализирана целосна игра tower defense со едно ниво и пет бранови, во која вештачката интелигенција е јасно одвоена и лесно за демонстрација: непријателите се движат по најкратка патека пронајдена со сопствена имплементација на A*, патеката се пресметува одново при секоја промена на мрежата, а кулите бираат цел според заменливи стратегии.

Правилото за блокирана патека покажува како може безбедно да се „пробува“ идна состојба: поставувањето се симулира без да се активираат настаните, па играта останува конзистентна. Безбедното ракување со подвижни цели (проектили што летаат до последната позната позиција, непријатели што умираат точно еднаш) ги избегнува типичните грешки на играта.

Балансирањето со автоматски играчи дава мерлив доказ дека повторното пребарување е стратегија: sидови што ја продолжуваат патеката во просек остават повеќе животи од кули покрај права патека, иако ефектот е мал и шумен. Резултатот е компактна основа за проучување на вештачката интелигенција во игри.

Изворен код: github.com/mircheskiangel/TowerDefence

Главна сцена: Assets/Scenes/TowerDefense.unity
Главни скрипти: Assets/Scripts
Генератор: Assets/Editor/LevelBuilder.cs